挑战杯 • 揭榜挂帅　　华为云码道 CodeArts • 题目 XH-202618

基于自研 AsNumpy



NumPilot

面向科学智能的昇腾 NPU 代码自动迁移智能体

Automatic Migration of NumPy Scientific Computing to Ascend NPU

NumPy 科学计算 / 网络 → NumPilot：AI 多智能体自动迁移 → 昇腾 NPU (AsNumpy • CANN)

作品信息

作品名称

NumPilot：面向科学智能的昇腾 NPU 代码自动迁移智能体

作品简介

NumPilot 是面向科学智能（AI for Science）的代码迁移智能体，将 NumPy 科学计算程序（遥感影像、物理仿真、基因分析等）及 NumPy 实现的网络自动迁移到国产昇腾 NPU，推动科学计算信创替代。系统以本团队自研、具有自主知识产权的 AsNumpy 加速库为底座，采用多智能体协同架构，由编排智能体统筹，分析、决策、代码迁移、算子生成、验证评测等多个智能体分工协作：大模型完成 np→ap 改写；遇缺失算子时自动生成 Ascend C 算子，经编译、单测、数值校验后注册回加速库，越用越全；并基于成本模型智能决策 CPU/NPU 卸载，保证数值一致与性能不退化。系统全程基于华为云 CodeArts 开发并部署于华为云。

科学智能 | AsNumpy 自主IP | 多智能体协同 | AI 算子生成 | CodeArts 开发 | 部署华为云

需求分析

目标：基于自研 **AsNumpy**（本组自主知识产权），把 NumPy 科学计算代码 / 网络自动迁移到昇腾 NPU —— 全程由大模型完成，近乎零改动、数值一致

现状：手工迁移的痛点

① 逐个改写 API（np.* → ap.*），上百个接口需人工核对

② 手工插入 CPU↔NPU 数据搬运，放错即拖垮性能

③ 小张量在 NPU 反而更慢，难判断算子去留（性能交叉点）

④ **AsNumpy 算子覆盖有缺口**，缺失算子需人工写 Ascend C，门槛极高

系统功能需求

自动迁移：大模型完成 np→ap 改写（AOT / JIT 双模）

AI 算子生成 ★：缺失算子→大模型生成 Ascend C 并接入 AsNumpy

智能卸载：成本模型按张量规模决定 CPU / NPU

一致性校验：迁移前后相对误差，根据不同数据类型决定

性能评测：输出加速比与迁移报告

迁移流程（端到端，全程大模型驱动）



背景工具---AsNumpy简要介绍



AsNumpy

面向昇腾NPU的NumPy库

license Apache 2.0 | python 3.9+ | CANN 8.2RC1+ | platform Ascend 910B | version 0.2.0

```python
import numpy as np
import asnumpy as ap

# Generate data on CPU
m1 = np.random.normal(0, 1, (3000, 3000)).astype(np.float32)
m2 = np.random.normal(0, 1, (3000, 3000)).astype(np.float32)

# Transfer to NPU
m1_npu = ap.ndarray.from_numpy(m1)
m2_npu = ap.ndarray.from_numpy(m2)

# Compute on NPU — same API as NumPy
result = ap.multiply(m1_npu, m2_npu)

# Transfer back to CPU
print(result.to_numpy())
```

https://gitcode.com/cann/asnumpy

AsNumpy是一款面向昇腾NPU的轻量级科学计算Python库，完全兼容NumPy API 它通过pybind11绑定层封装华为CANN算子，并借助与numpy.ndarray镜像一致的NPUArray数据结构对外暴露接口

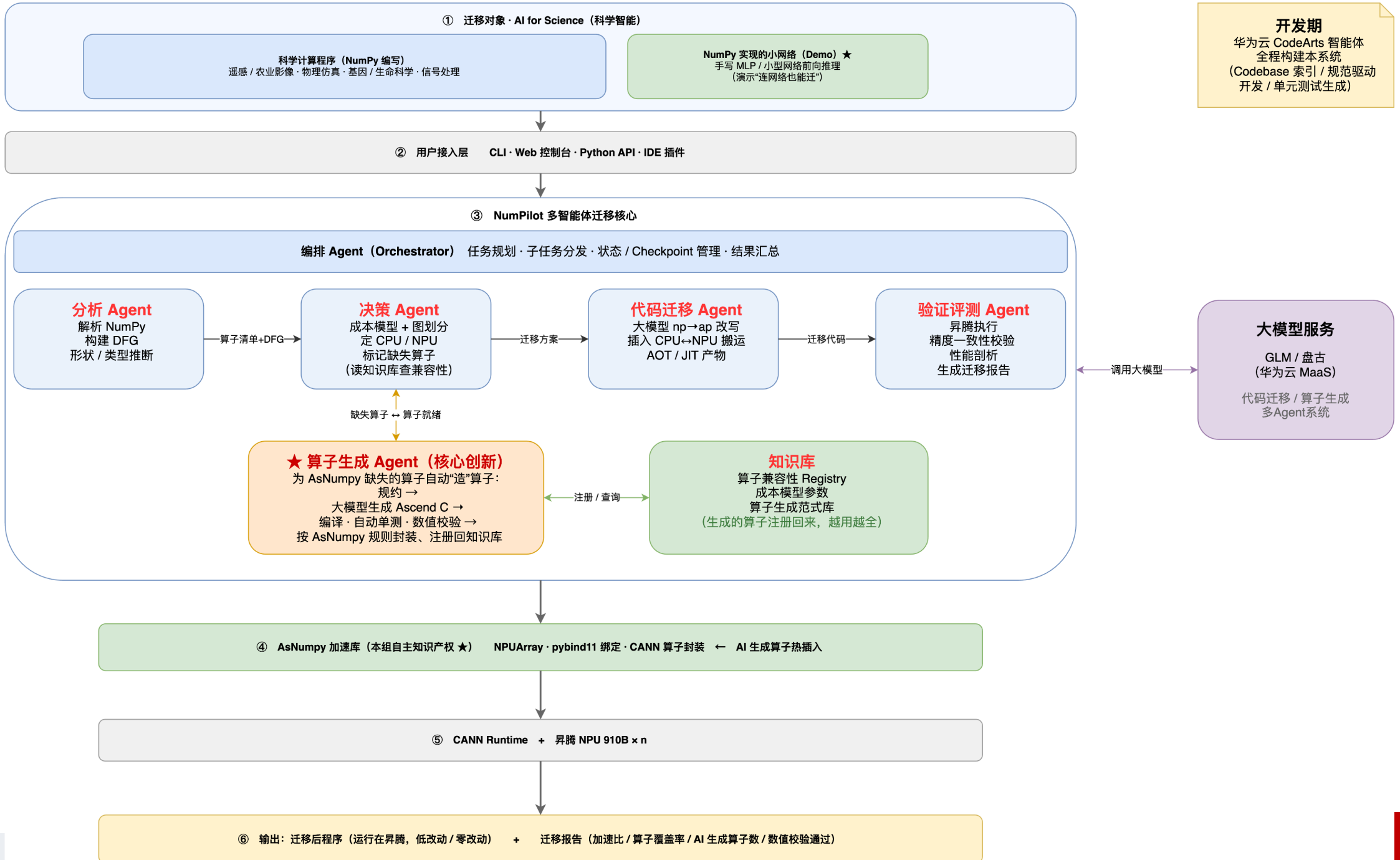
- **兼容 NumPy 的 API** — 函数名称和签名与 NumPy 保持一致；只需少量修改即可迁移现有代码
- **昇腾 910B 原生加速** — 算子通过 CANN ACLNN 直接在 NPU 上运行，无框架开销
- **自动资源管理** — NPUArray 析构函数自动释放设备内存（RAII 机制）
- **双向数据传输** — 提供 from_numpy() 和 to_numpy() 实现 CPU 与 NPU 之间的无缝数据交换
- **广播支持** — 内置 GetBroadcastShape() 函数遵循 NumPy 广播语义
- **算子可扩展性** — 缺失的 CANN 算子通过按照规范格式进行自定义Ascend C算子开发进行补充

性能表现

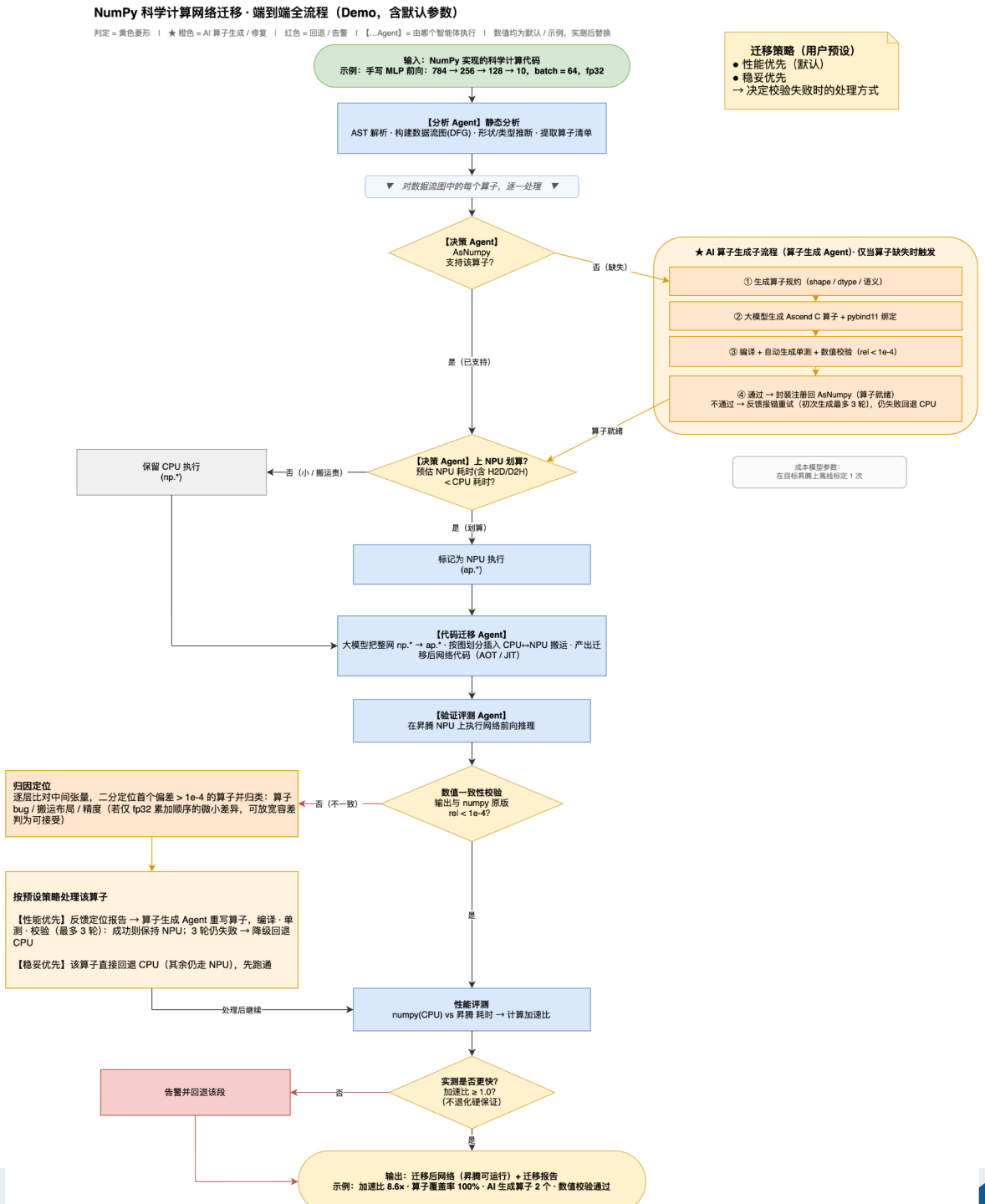
在 3000×3000 的 `float32` 数据上，在相同机器上，`ap.multiply()` 比 `np.multiply()` **快 128.70 倍**。

形状	AsNumpy (NPU)	NumPy (CPU)	加速比
(500, 500)	1.9355 秒	0.1708 秒	0.09 倍
(1000, 1000)	0.0692 秒	0.7029 秒	10.16 倍
(2000, 2000)	0.1033 秒	3.8387 秒	37.17 倍
(3000, 3000)	0.1115 秒	14.3567 秒	**128.70 倍**

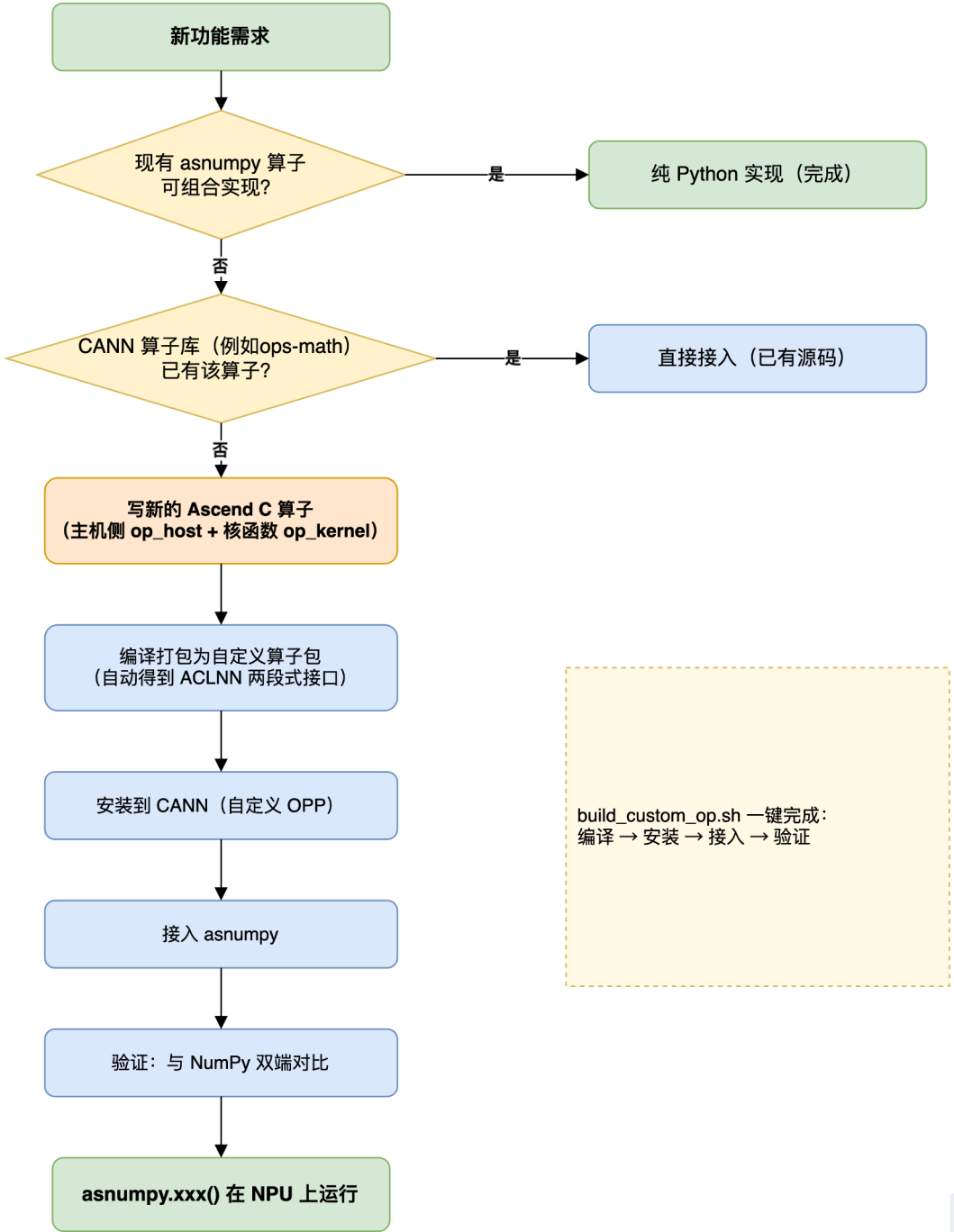
NumPilot 整体架构：基于 AsNumpy 的 AI for Science 昇腾自动迁移系统



矢量图
可放大查看

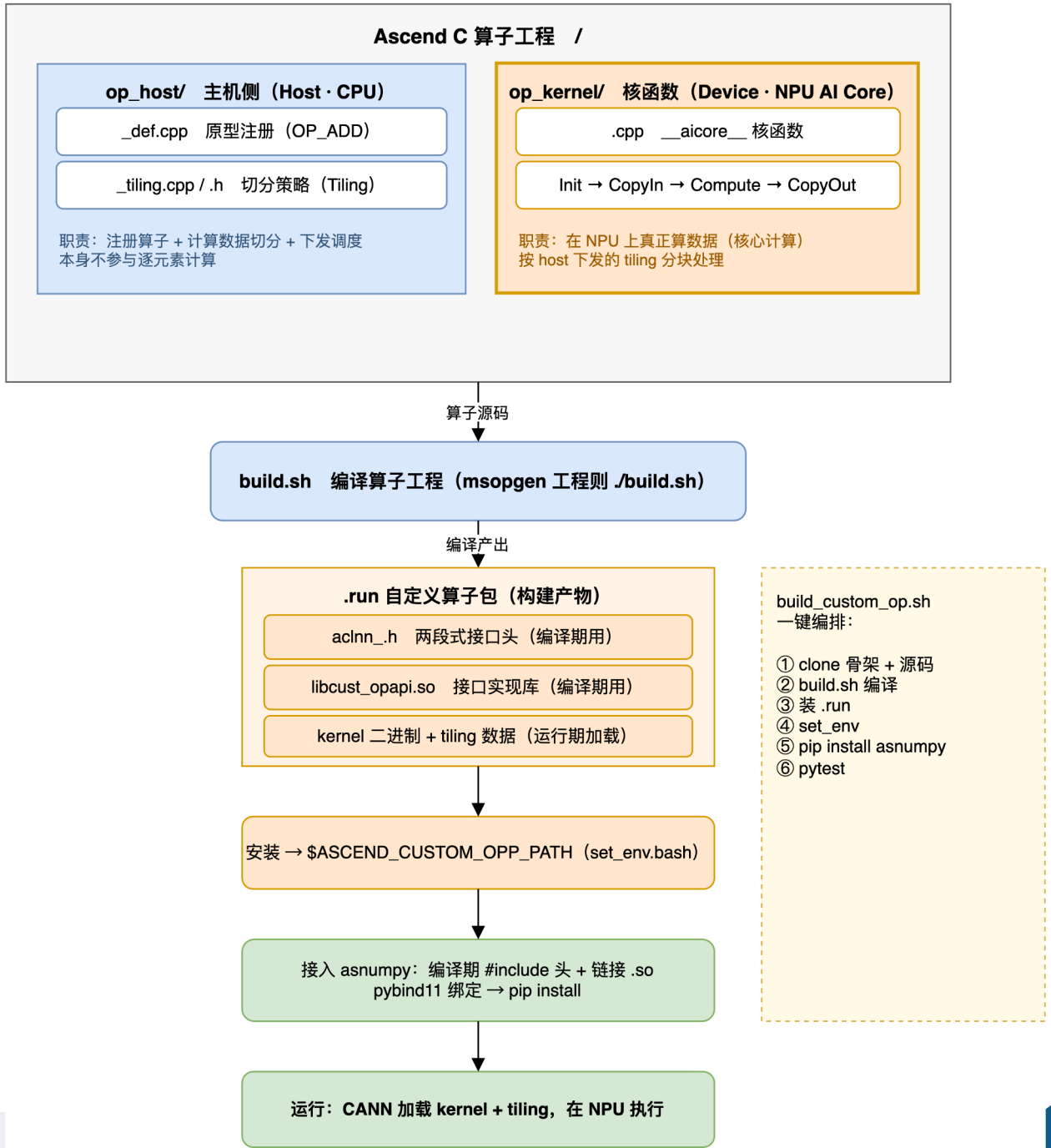

NumPy 科学计算网络迁移 · 端到端全流程（Demo，含默认参数）
判定 = 黄色菱形 | ★ 橙色 = AI 算子生成 / 修复 | 红色 = 回退 / 告警 | 【…Agent】= 由哪个智能体执行 | 数值均为默认 / 示例，实测后替换
输入：NumPy 实现的科学计算代码
示例：手写 MLP 前向：784 → 256 → 128 → 10，batch = 64，fp32
迁移策略（用户预设）
● 性能优先（默认）
● 稳妥优先
→ 决定校验失败时的处理方式
【分析 Agent】静态分析
AST 解析 · 构建数据流图(DFG) · 形状/类型推断 · 提取算子清单
▼ 对数据流图中的每个算子，逐一处理 ▼
【决策 Agent】
AsNumpy
支持该算子?
否（缺失）
★ AI 算子生成子流程（算子生成 Agent）· 仅当算子缺失时触发
① 生成算子规约（shape / dtype / 语义）
② 大模型生成 Ascend C 算子 + pybind11 绑定
③ 编译 + 自动生成单测 + 数值校验（rel < 1e-4）
④ 通过 → 封装注册回 AsNumpy（算子就绪）
不通过 → 反馈报错重试（初次生成最多 3 轮），仍失败回退 CPU
是（已支持）
算子就绪
【决策 Agent】上 NPU 划算?
预估 NPU 耗时(含 H2D/D2H)
< CPU 耗时?
否（小 / 搬运费）
保留 CPU 执行
(np.*)
成本模型参数：
在目标昇腾上离线标定 1 次
是（划算）
标记为 NPU 执行
(ap.*)
【代码迁移 Agent】
大模型把整网 np.* → ap.* · 按图划分插入 CPU↔NPU 搬运 · 产出迁移后网络代码（AOT / JIT）
【验证评测 Agent】
在昇腾 NPU 上执行网络前向推理
数值一致性校验
输出与 numpy 原版
rel < 1e-4?
否（不一致）
归因定位
逐层比对中间张量，二分定位首个偏差 > 1e-4 的算子并归类：算子 bug / 搬运布局 / 精度（若仅 fp32 累加顺序的微小差异，可放宽容差判为可接受）
按预设策略处理该算子
【性能优先】反馈定位报告 → 算子生成 Agent 重写算子，编译 · 单测 · 校验（最多 3 轮）：成功则保持 NPU；3 轮仍失败 → 降级回退 CPU
【稳妥优先】该算子直接回退 CPU（其余仍走 NPU），先跑通
是
处理后继续
性能评测
numpy(CPU) vs 昇腾 耗时 → 计算加速比
实测是否更快?
加速比 ≥ 1.0?
(不退化硬保证)
否
告警并回退该段
是
输出：迁移后网络（昇腾可运行）+ 迁移报告
示例：加速比 8.6x · 算子覆盖率 100% · AI 生成算子 2 个 · 数值校验通过

AsNumpy 补充新功能 — 全流程



build_custom_op.sh 一键完成：
编译 → 安装 → 接入 → 验证

Ascend C 算子工程架构（op_host vs op_kernel）与接入关系



NumPilot 与传统方案的对比

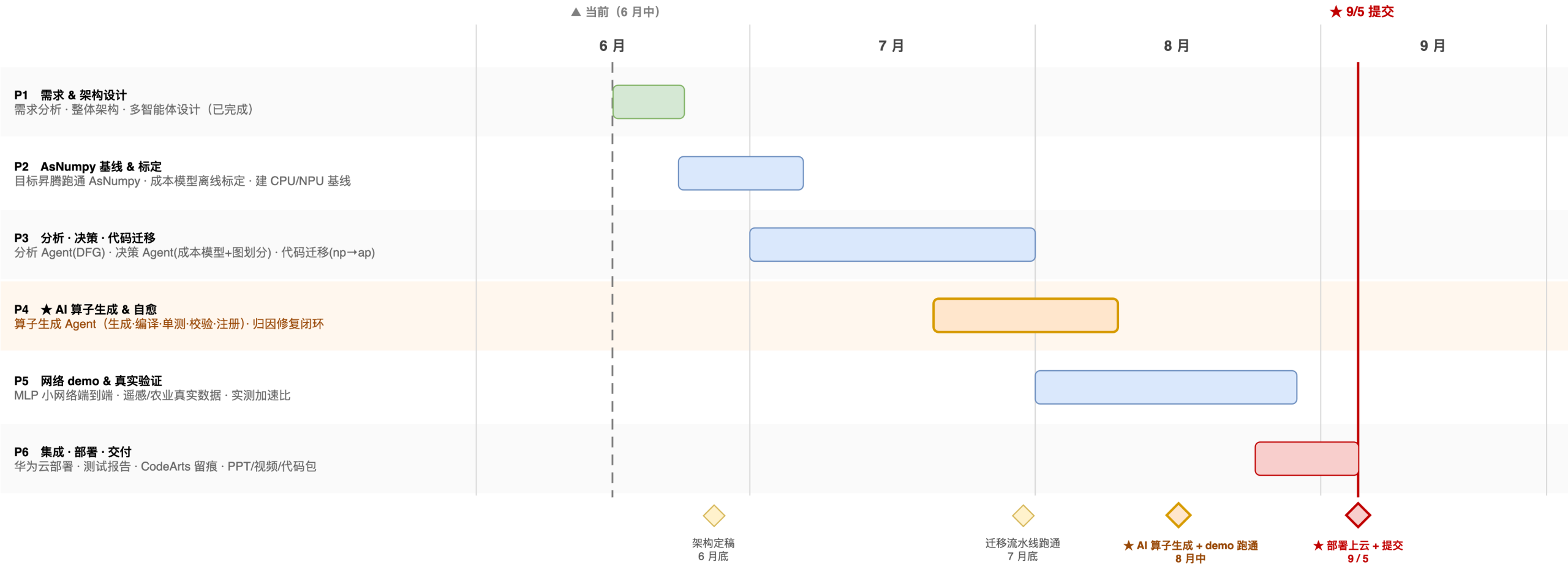
NumPilot 差异化对比 —— 为什么现成方案替代不了

维度	手工迁移 （直接用 AsNumpy）	CuPy （NumPy on GPU）	torch_npu / MindSpore （框架方案）	★ NumPilot （本作品）
面向硬件 / 生态	昇腾 NPU（国产）	NVIDIA GPU（CUDA）	昇腾 NPU（国产）	**昇腾 NPU（国产 · 信创替代）**
适用负载	NumPy 科学计算	NumPy 科学计算	深度学习网络（须框架重写）	**NumPy 科学计算 + numpy 网络**
迁移改动量	大：逐个改 API、手工插搬运	中：np→cp，仍需改代码	大：须用框架重写	**近乎零：大模型自动改写（AOT / JIT）**
缺失算子处理 ★	人工写 Ascend C（门槛极高）	回退 CPU 或自写 CUDA	受限于框架算子库	**★ AI 自动生成 Ascend C，校验后注册，越用越全**
CPU / NPU 卸载决策	人工判断，易拖垮性能	全 GPU / 手动	框架 / 人工	**成本模型 + 图划分自动决策**
数值一致 & 性能保障	自行验证	自行验证	自行验证	**自动校验（容差按 dtype）+ 归因自愈 + 不退化**
开发方式 / AI 原生 ★	纯人工	纯人工	纯人工	**★ 全程大模型多智能体 + CodeArts 开发**

要点：把 CuPy 式「NumPy 低改动迁移」带到国产昇腾，并叠加 AI 自动补算子、成本模型智能卸载、数值自愈与不退化 —— 这一组合是手工迁移 / CuPy / 框架方案都不具备的。

开发节奏（拟定）

NumPilot 技术路线与里程碑（2026 年 6 月 – 9 月 5 日提交）



说明：① 各阶段全程基于华为云 CodeArts 智能体开发并留存使用记录（对应完整性 50 分）；阶段之间适度并行。 ② 提交后：9/20 前完成初审，11 月终审擂台赛。